UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Lesson 2: Data Exploration and Preparation

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

2 October 2026

Before we start

Exercise 1 was set last week. We discuss it now.

- The wine-quality workflow, end to end
- What counts is **methodology**, not accuracy
- A show of hands on how you framed it

What today builds on

- Lesson 1’s rule: **nothing is learned before the split**
- Lesson 1’s tool: `Pipeline`, so the rule is structural, not just discipline
- Today: apply both to data that is actually messy

Today

- Exploratory analysis
- Missing values
- Outliers
- Scaling and encoding
- Feature engineering and pipelines
- Leakage, in a form nothing warns you about

One dataset, all lesson long

2000 synthetic telecom customers. Will they **churn**?

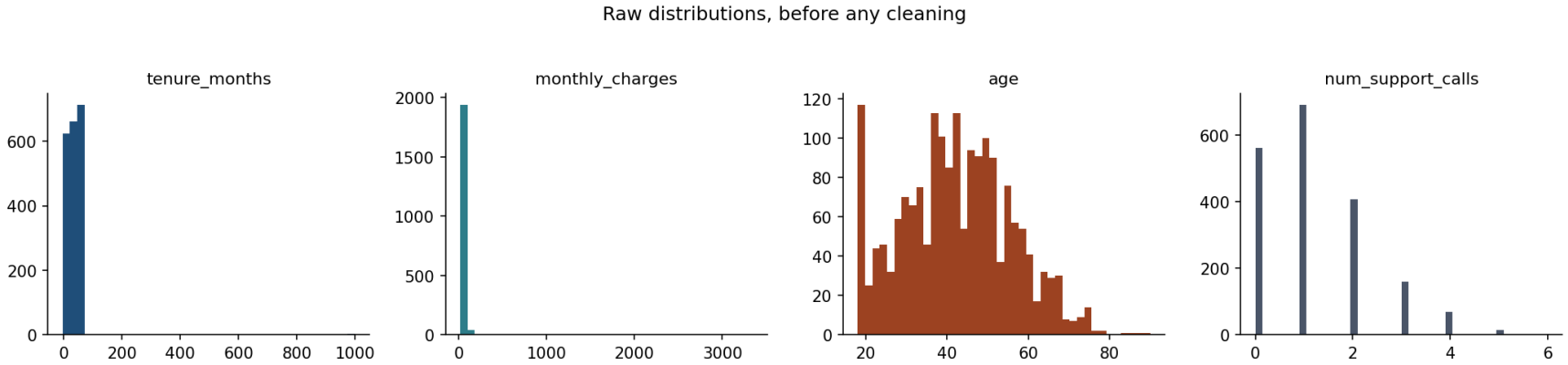
- Numeric: tenure, monthly charges, age, support calls
- Categorical: contract type, region, zip code (493 levels)
- Missing values, outliers, and one column built to carry **no signal**

Look before you touch anything

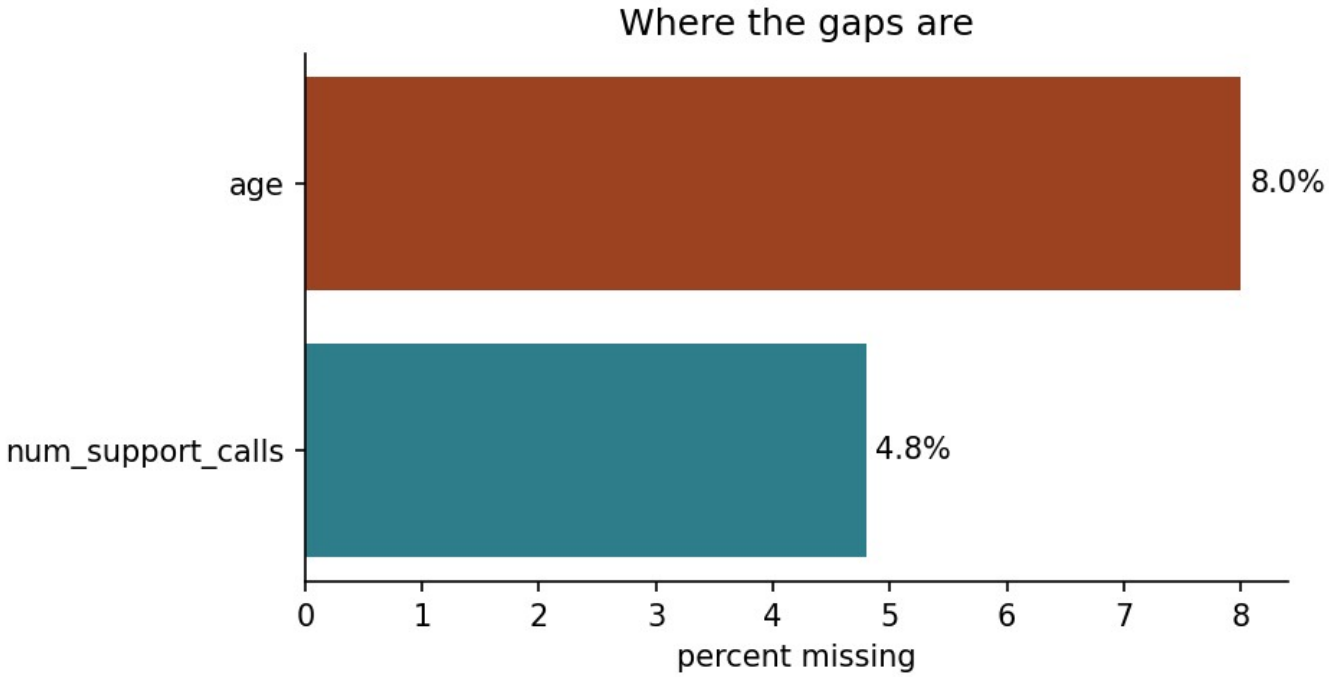
`.info()`, `.describe()`, class balance, first: always.

- 2000 rows, 8 columns, mixed types
- Churn rate: **19.4%** → baseline accuracy 80.6%
- `tenure_months` max: 999. Not a typo in the slide.

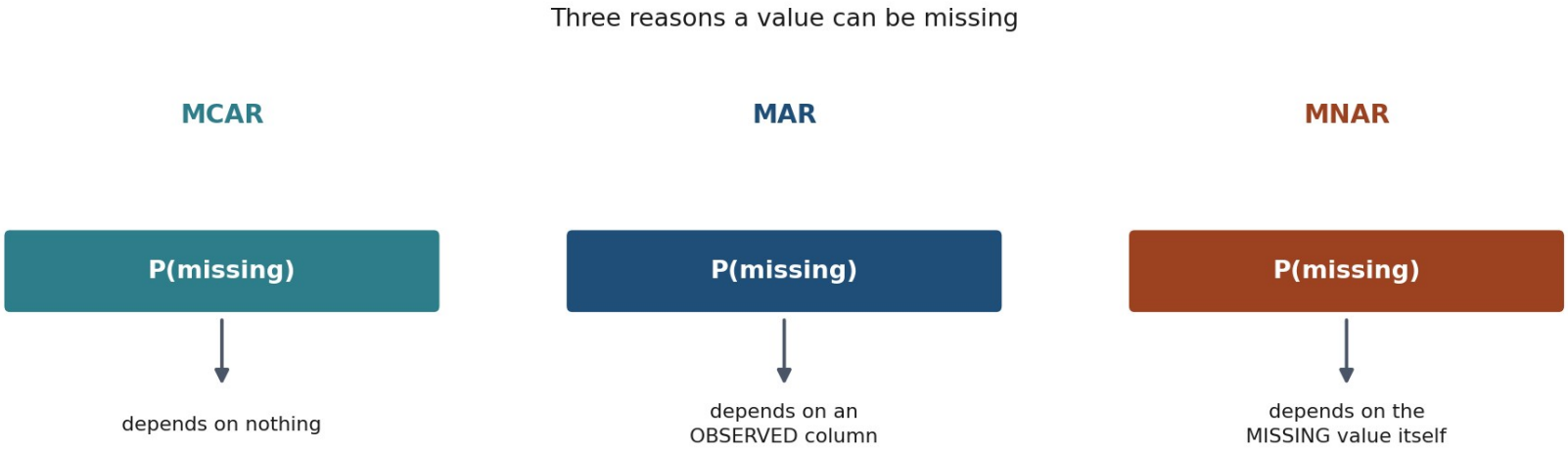
Two of these four have no shape left to read



Where the gaps are



Three reasons a value is missing



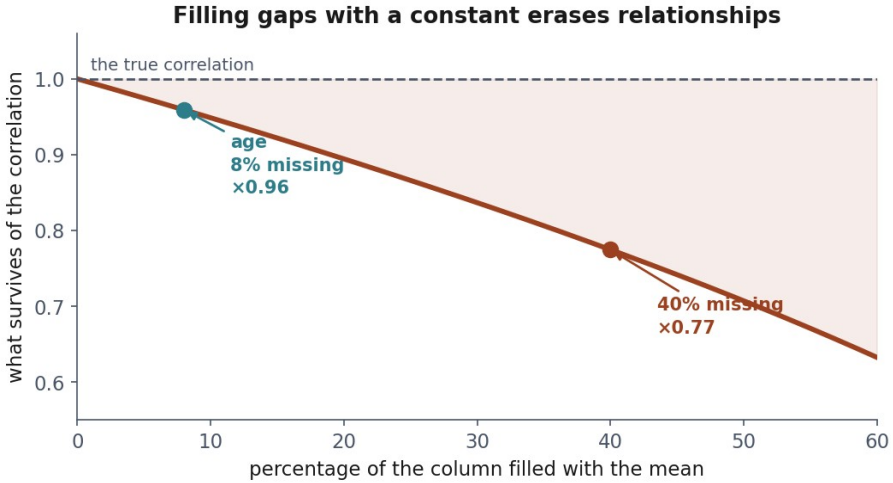
age: MCAR

Some customers just left it blank. Nothing systematic.

- Safe to impute with a fixed statistic
- The bias question is not “is this fair”: it is “what does filling it cost”

What mean imputation actually costs

Every correlation involving the imputed column shrinks, even under MCAR.



num_support_calls: MAR

Missing more often for long-tenure customers, not by chance.

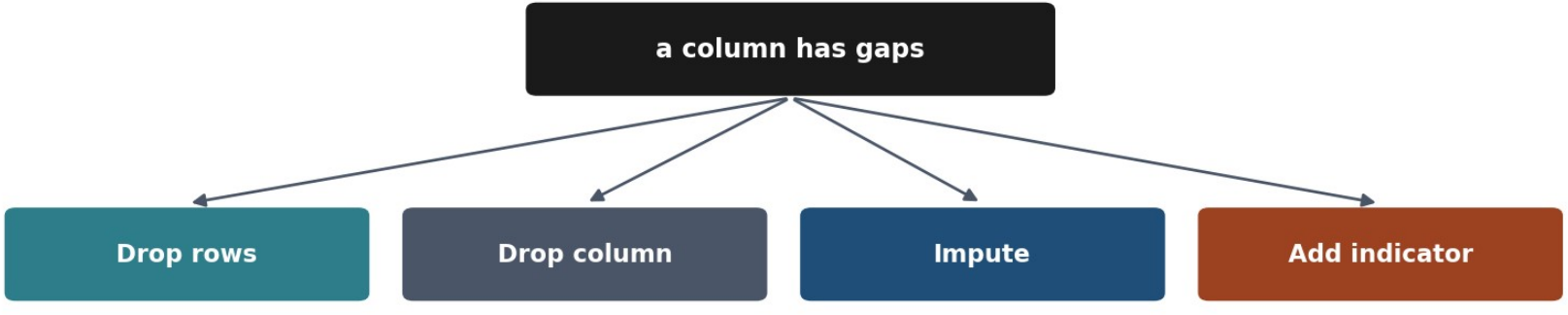
- Treating it as MCAR reweights the sample the wrong way
- A **missingness indicator** column preserves the signal even after filling

What to do about a gap

- **Drop rows**: only if few, and only if MCAR
- **Drop the column**: if it is missing too often to help
- **Impute**: mean/median, most-frequent, `KNNImputer`
- **Add a missingness indicator**: keeps MAR signal alive

Choosing among the four

Which one you pick is a question about why the data is missing



the mechanism decides, and every option except dropping the column learns a statistic, so it belongs inside the training fold

An outlier is far from the rest - not necessarily wrong

- `tenure_months` runs **-3 to 999**; ordinary customers sit at 0 to 72
- Three reasons: a **recording error**, a **rare but genuine** value, a **different population**
- The arithmetic cannot tell which: 3,344.7 may be a typo or a corporate account
- Two rules follow - standard deviations, then quartiles - and both only **flag**

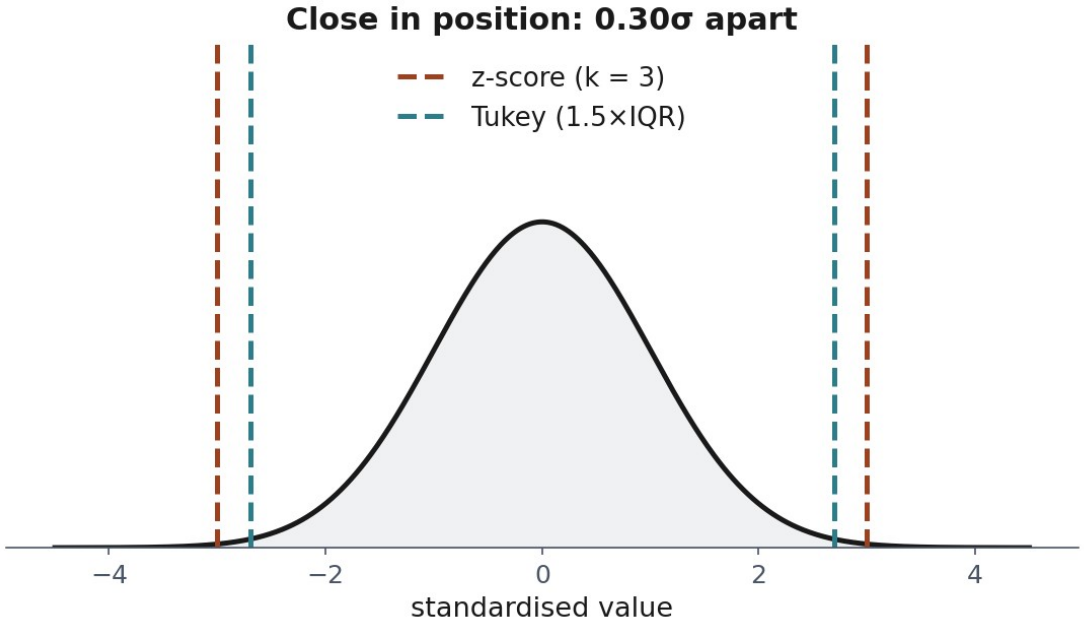
Rule 1: distance in standard deviations, flag beyond k = 3

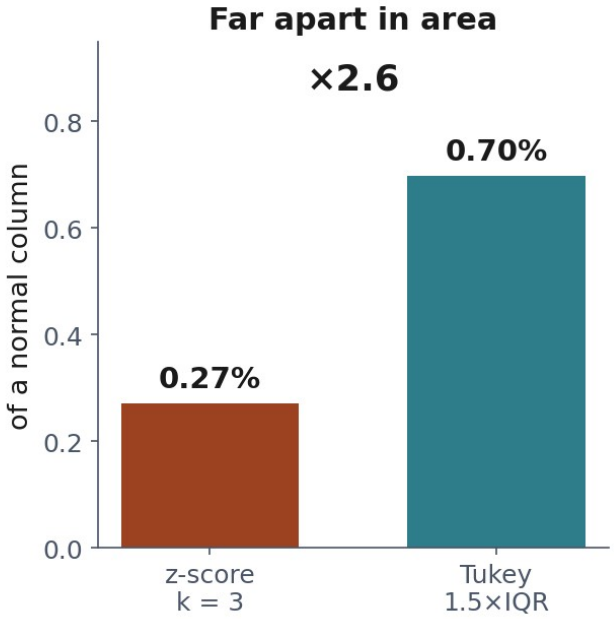
$$z_i = \frac{x_i - \bar{x}}{s}$$

Rule 2: distance measured in quartiles

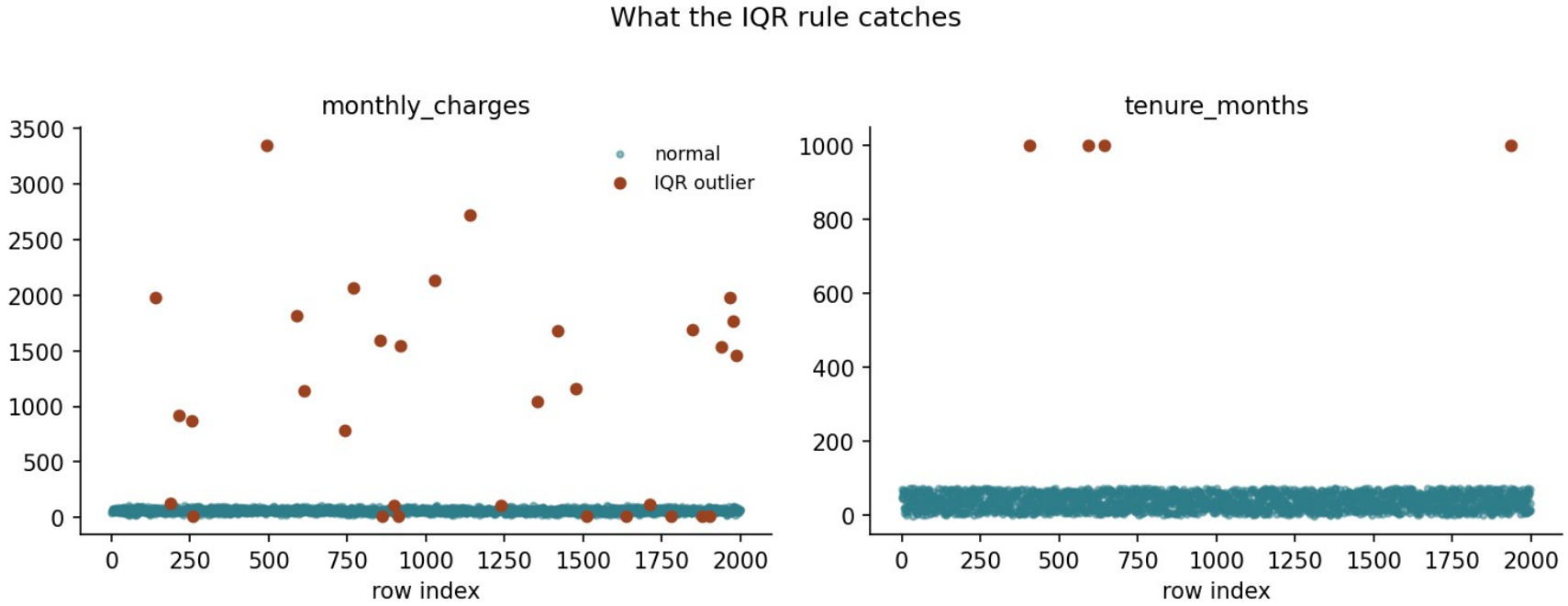
$$[\ Q_1 - 1.5\ \text{IQR},\quad Q_3 + 1.5\ \text{IQR}\ ]$$

Same neighbourhood, not the same answer





The robust rule is the one that got it wrong



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What neither rule can tell you

`tenure_months` of **-3** is impossible. Neither rule flags it.

- -3 is not *extreme* relative to a column spanning 0–72
- It is not *unusual*. It is *invalid*: a different question entirely

Once something is flagged

Detection finds candidates. What to do next is a **domain** decision.

- **Cap** it at the fence: 3,344.7 becomes 110.0 (*Winsorising*)
- **Remove** the row
- **Correct** it, if the true value is recoverable
- **Leave** it: unusual is not the same as wrong

Notebook 1, live

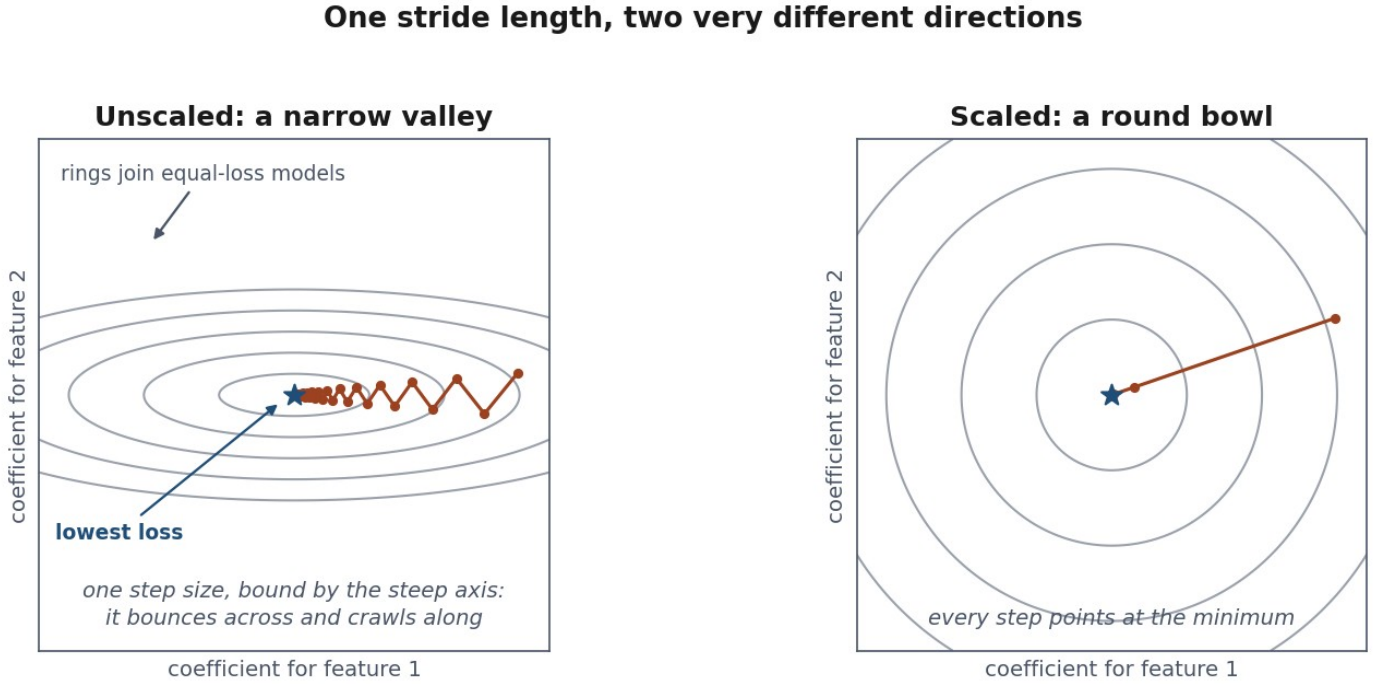
Exploration, missingness mechanisms, both outlier rules: from scratch.

Break

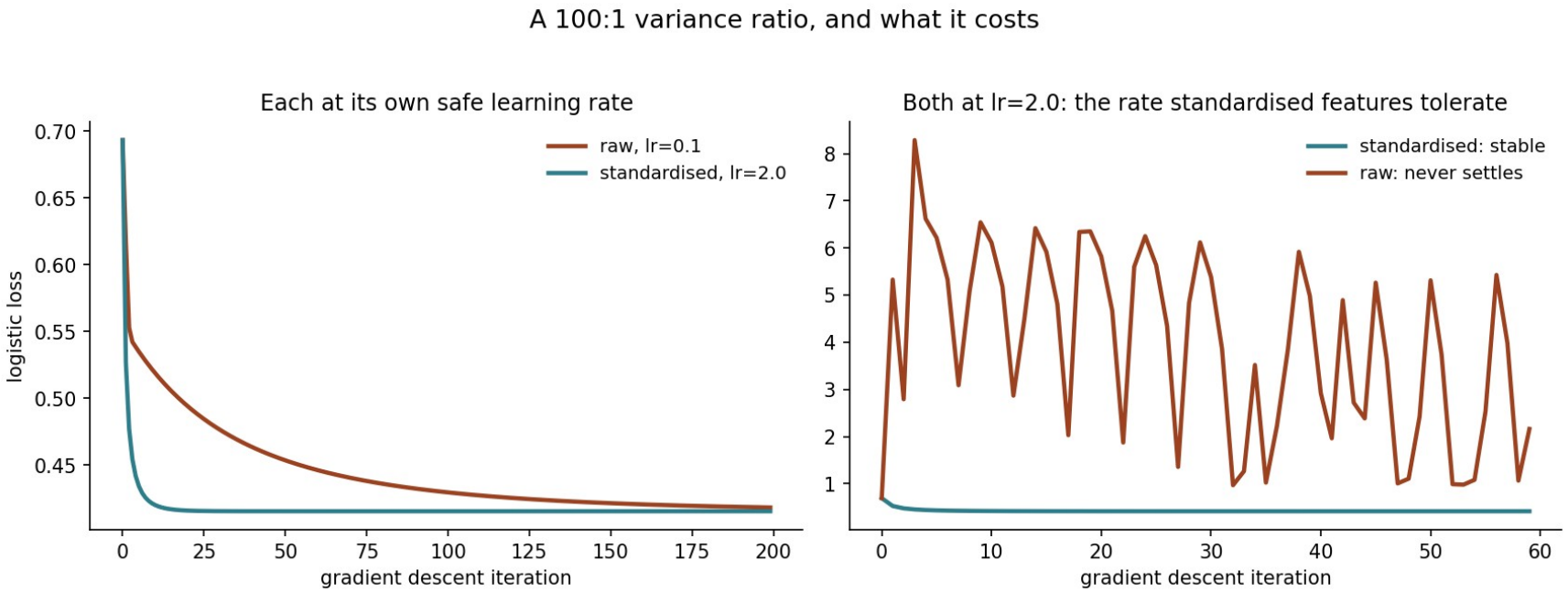
Why scale?

- `tenure_months`: 0–72. `monthly_charges`: 15–128
- **Training** = choosing the model's numbers to make Lesson 1's average loss small
- **Gradient descent** does it by repeated small steps downhill on that loss
- One step size, along every axis, every iteration. Can it suit both columns?

A ravine, not a bowl

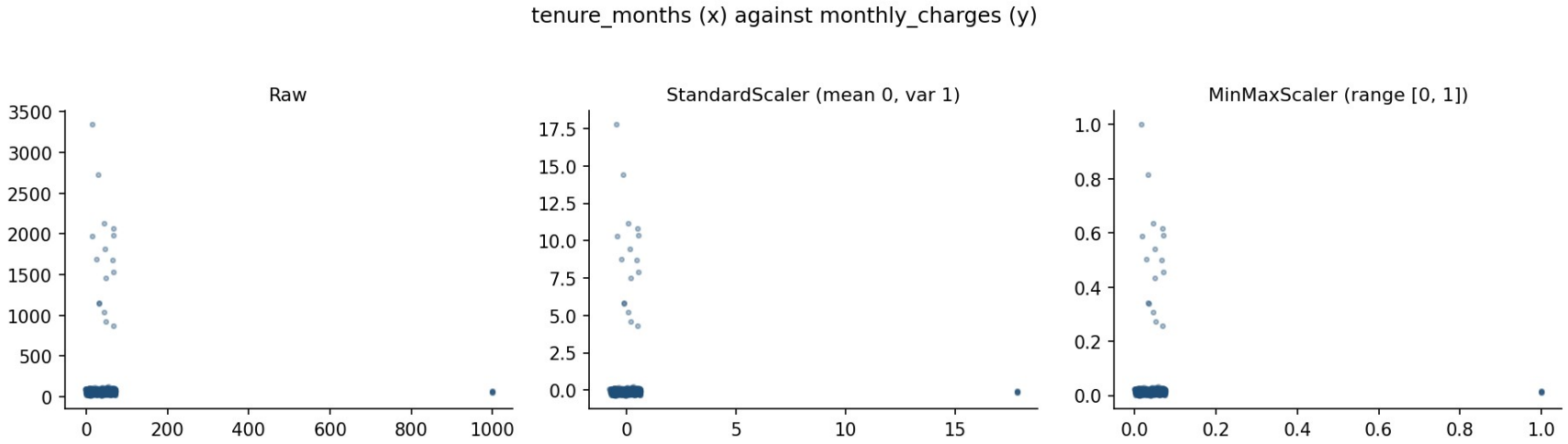


A 100:1 variance ratio, and what it costs



StandardScaler vs MinMaxScaler

Two standard choices, both fitted on training data only.



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Every encoding makes a claim about the categories

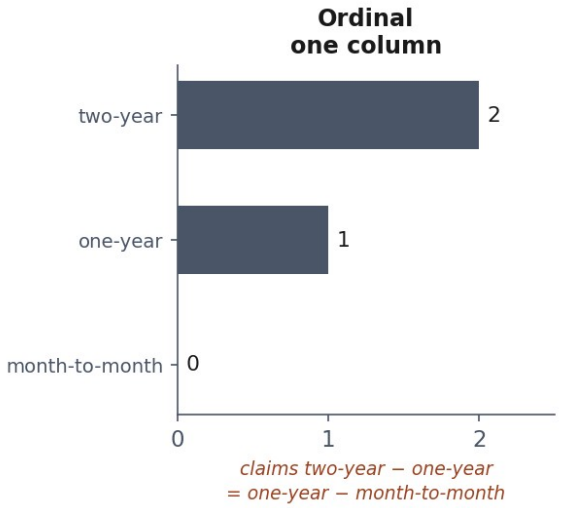
A model does arithmetic, and `"month-to-month"` offers none.

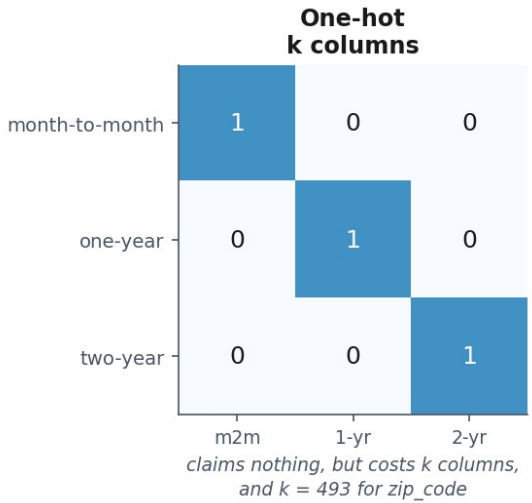
contract_type	rows	churn rate
month-to-month	1,092	0.266
one-year	488	0.137
two-year	420	0.071

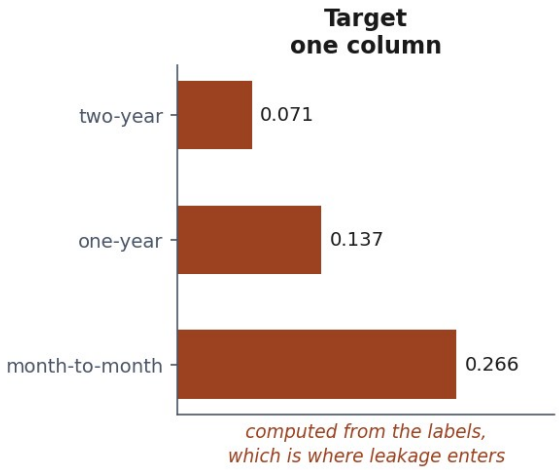
One customer, three encodings

Encoding	A one-year customer becomes	What that asserts
One-hot	`[0, 1, 0]`	simply different: no order, no distances
Ordinal	`1`	ordered, **and** equally spaced
Target	`0.137`	the level's own average outcome stands in for it

Equal steps, unequal meaning







One-hot claims nothing, and pays in columns

- A column with k levels becomes k binary columns, one 1 per row
- It asserts no order and no distance - which is why it is the default
- The price is width: `zip_code` would add **493** columns to 2,000 rows
- Target encoding compresses any cardinality into one column - computed from the labels, which is where leakage gets in

All k dummies plus an intercept cannot both be free

Most models carry a column of ones - the **intercept** - so they can fit a baseline level.

$$\sum_{j=1}^{k} \text{dummy}_j = 1 = \text{intercept}$$

Rank deficient by exactly one

Why all k dummies plus an intercept cannot work

contract_type	m2m	1y	2y	sum	intercept
month-to-month	**1**	0	0	**= 1**	**1**
one-year	0	**1**	0	**= 1**	**1**
two-year	0	0	**1**	**= 1**	**1**

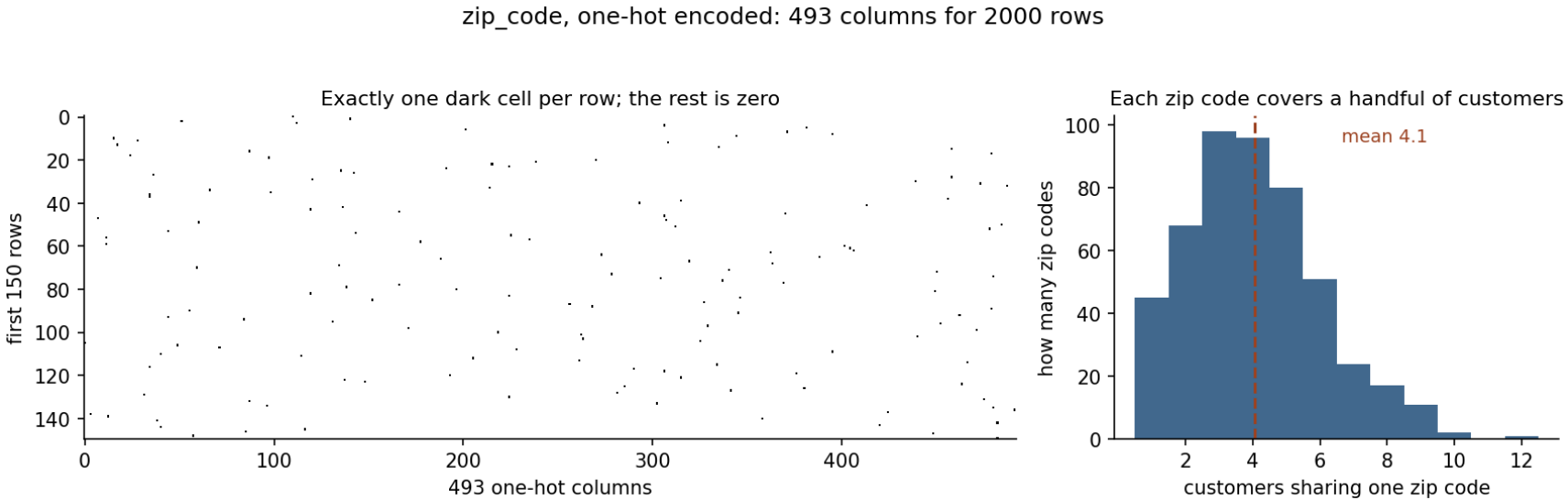
the k dummy columns always sum to the intercept column: rank deficient by exactly one

The curse of dimensionality, briefly

One-hot `zip_code`: **493** new columns, mostly zero.

- Distance-based methods (Lesson 6): everything looks equidistant
- More parameters to estimate, same sample size → more variance

zip_code: 493 codes, 2,000 customers

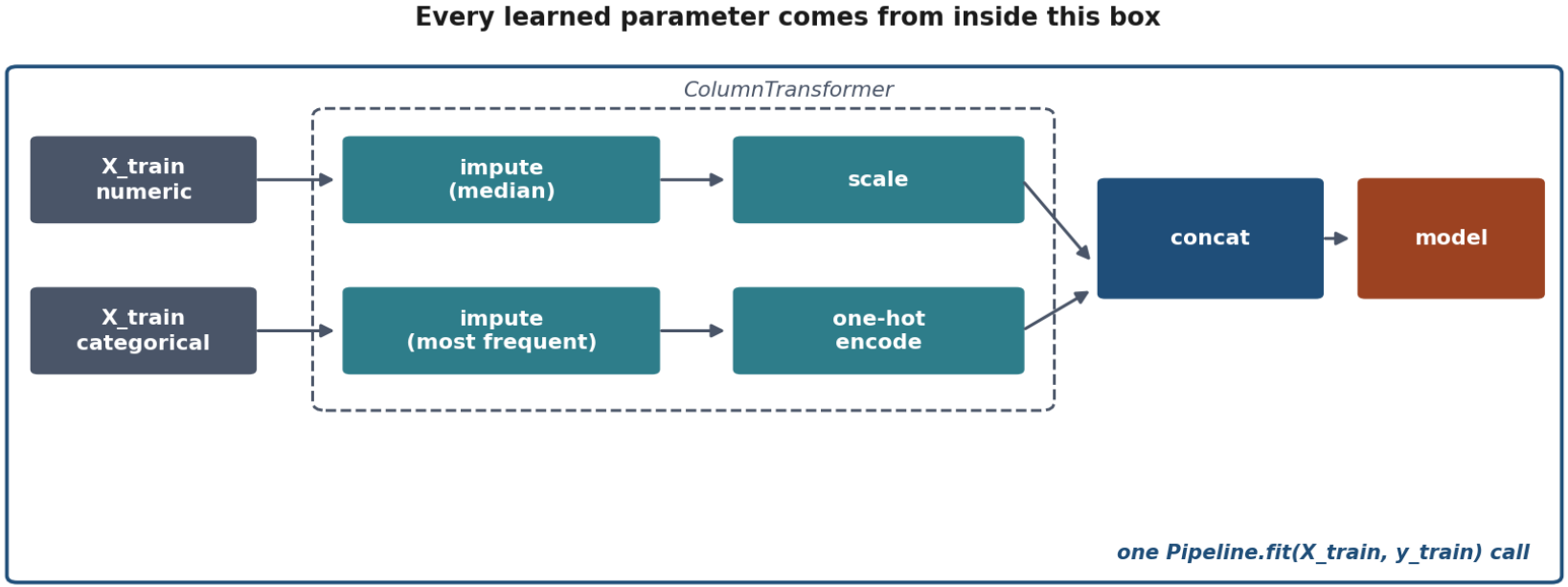


Restating Lesson 1’s argument

f includes **every** learned parameter, not just “the model part”.

$$\mathrm{E}_{T \sim \mathcal{D}^m}\left[\hat{R}_T(f)\right] = R(f) \quad \text{only if } f \text{ is independent of } T$$

ColumnTransformer + Pipeline



The code

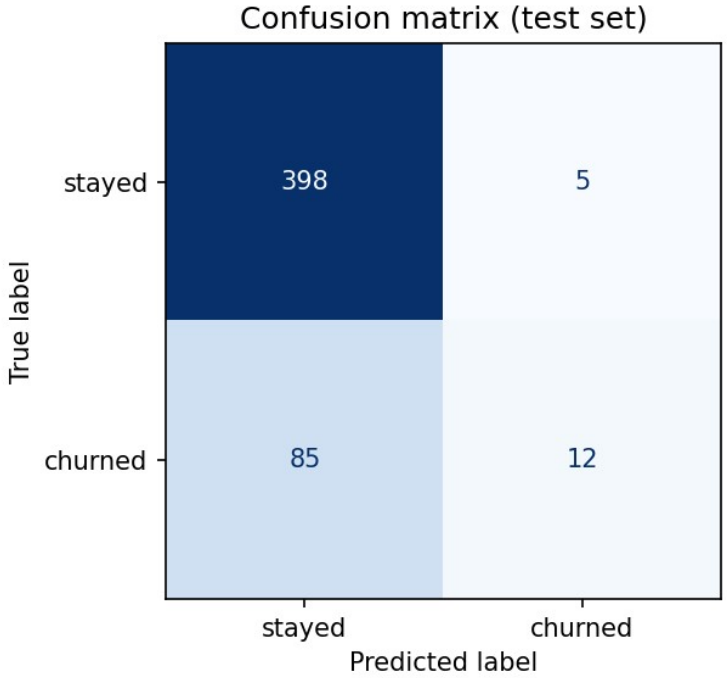
```
prep = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("cat", cat_pipe, cat_cols),
])
model = Pipeline([
    ("prep", prep),
    ("clf", clf),
])
model.fit(X_train, y_train)
```

The model, on churn

- Baseline: **0.806**
- Model (numeric + the low-cardinality categoricals, no zip): **0.820** accuracy, **0.751** AUC (area under the receiver operating characteristic curve)

Modest accuracy gain. Why?

Where the errors fall



Feature engineering

Ratios, interactions, binning: a linear model cannot invent these itself. AUC: 0.7514 → 0.7542. **Under three thousandths: one split cannot tell that from noise.**

$$\text{charge\_per\_tenure} = \frac{\text{monthly\_charges}}{\max(\text{tenure\_months},\ 0) + 1}$$

Notebook 2, live

Scaling from scratch, the dummy trap, `ColumnTransformer`, `Pipeline`.

Same rule, broken three ways

Same rule, broken three ways: only the first one looks like a bug

Lesson 1	select features on the full dataset *visible: an obvious extra step*
Today, leak 1	impute a missing value on the full dataset *invisible: looks like ordinary cleaning*
Today, leak 2	encode a category on the full dataset *invisible: two unremarkable lines of code*

Leak 1: impute before splitting

`KNNImputer` fills a gap using **other rows' values**.

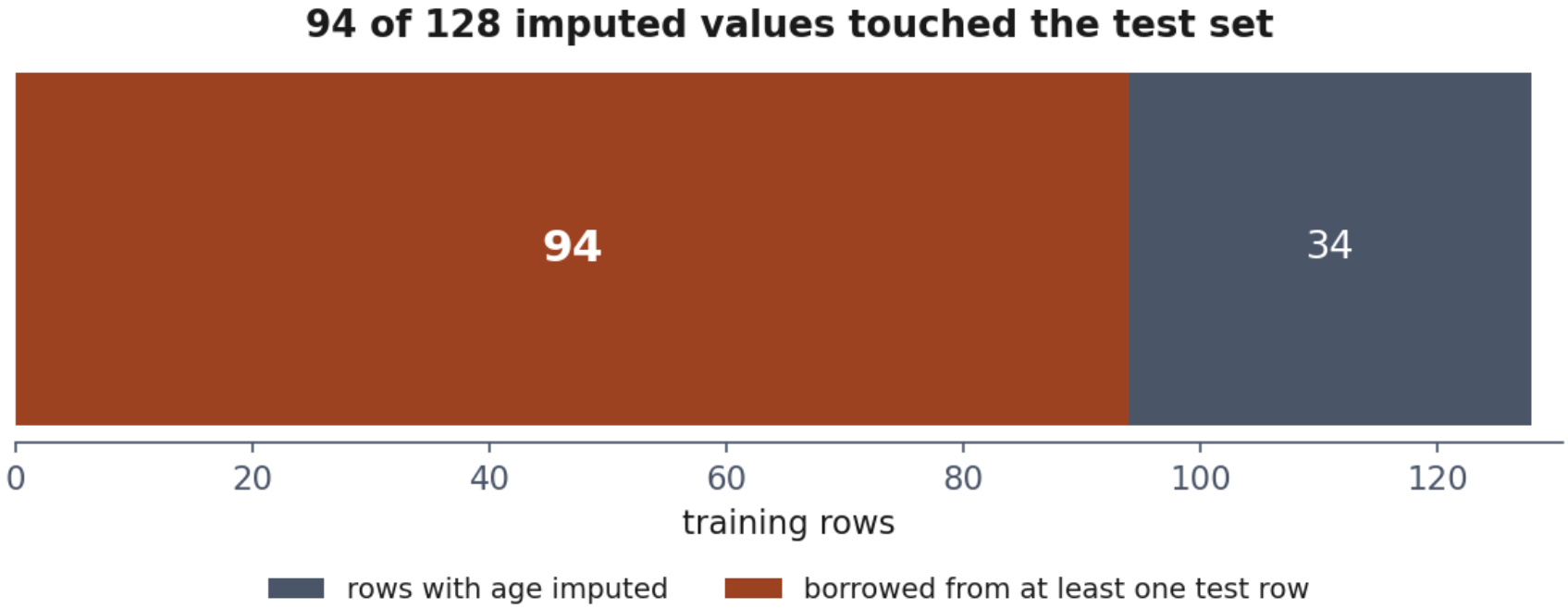
Fit it on train + test, and some of those rows are in the test set.

The smoking gun

94 of 128 training rows with missing age had a **test-set row** among the five donors used to fill it.

Nothing raised an error.

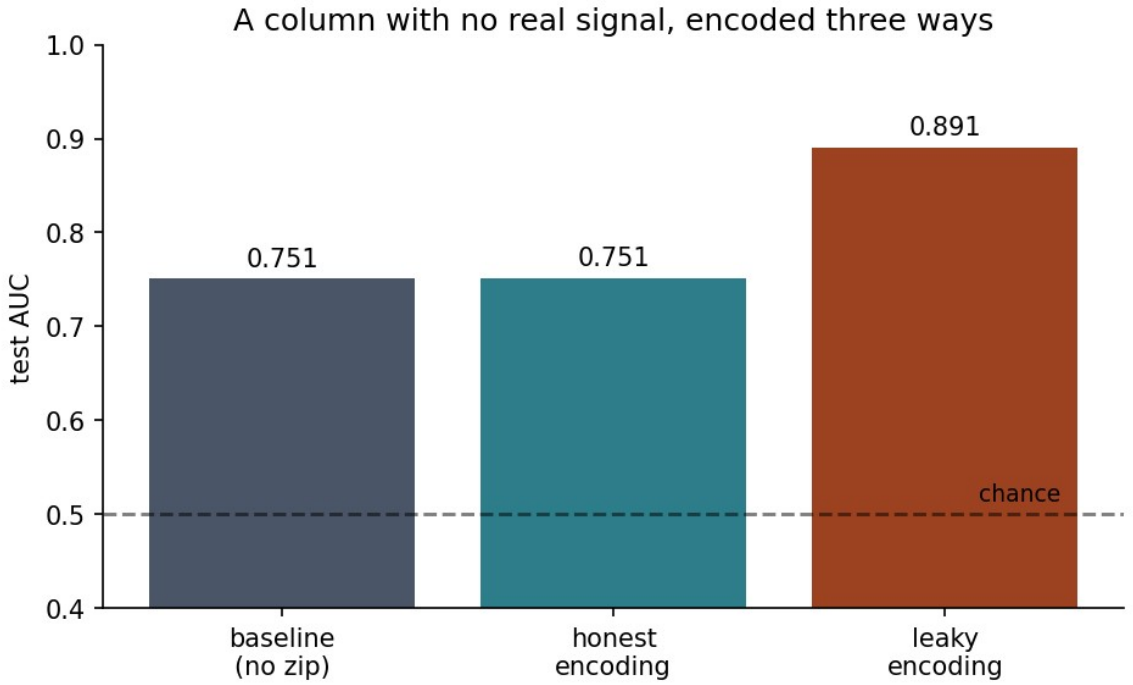
Leakage, counted



Leak 2: encode before splitting

Replace `zip_code` with the **average churn rate of its own customers**, computed before anybody has split anything.

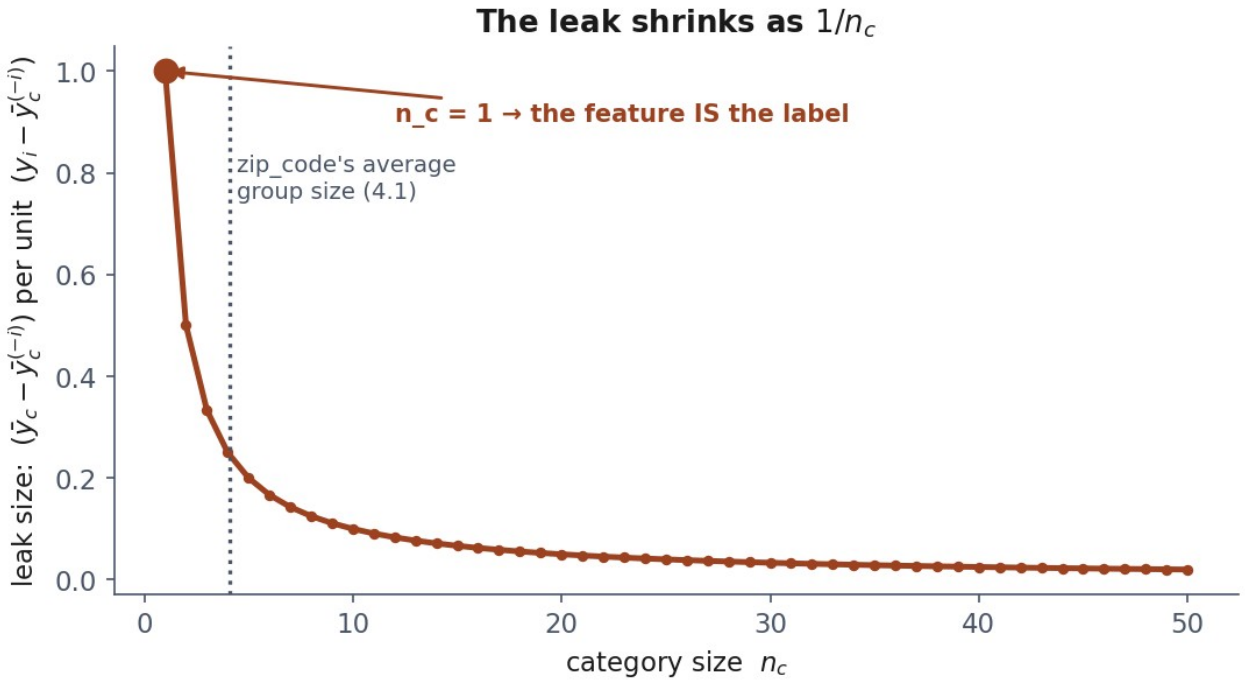
A column with no real signal, encoded three ways



Your own disagreement, divided by your group size

$$\bar{y}_c - \bar{y}_c^{(-i)} = \frac{y_i - \bar{y}_c^{(-i)}}{n_c}$$

The leak shrinks as 1/n_c



The fix

`sklearn.preprocessing.TargetEncoder`: cross-fitted inside the pipeline.

Each row’s encoded value comes from **other** folds, never its own label.

The rule, restated

Everything that **learns from data** belongs inside the fold it is fitted on.

A mean. A median. A set of neighbours. A per-category average. A set of bin edges.

Notebook 3, live

Trace the imputation leak. Watch the encoding leak manufacture 0.89 from noise.

What we did today

- Explored before transforming: types, gaps, shape, correlations
- Diagnosed *why* values are missing, not just *how many*
- Two outlier rules, and what neither can tell you
- Scaling, encoding, engineering: all inside `Pipeline`
- Two leaks that look nothing like leaking

Homework: we discuss it on Friday 9 October

`Exercises/02_data_exploration_and_preparation.md`

Build the full preparation pipeline yourself, and **find a leak on purpose** before fixing it.

Before next week

- Work the three notebooks in order
- Read the handout: the derivations are examinable
- Take the quiz
- **Do the exercise**

Next: regression, the first model we derive completely.